Vehicle Routing: A Disturbance Approach

Nick Young + Kyran Flynn

High-level Approach

- Run complete dynamic IP model on smallest instance.

- Otherwise, generate initial solution using one of:
  - Savings Algorithm (Clark-Wright)
  - Sweep Algorithm
  - Random Initial Feasible (CP CPLEX solver)

- Then, improve on the initial solution using one of:
  - Iteratively Disturbed Best Improvement
  - Simulated Annealing (Random Restarts)

Complete Search

- Modelled in CPLEX and solved using provided IP solver.
- Used *lazy* subtour elimination to avoid invalid solutions.

Factorial # of constraints :/

$$Z = \{(X_{ij}^v): \sum_{i \in B} \sum_{j \in B} X_{ij}^v \leqslant |B| - 1 \quad \text{for } B \subseteq V/\{0\}; |B| \geqslant 2\}.$$

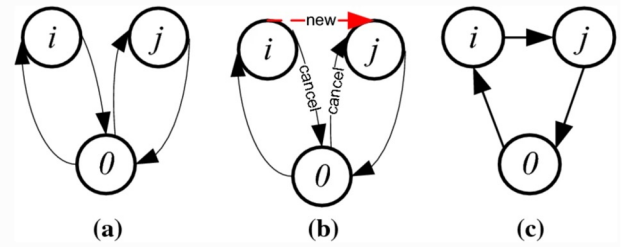
- Tried CPLEX '*LazyConstraint*' functionality, *very* slow…
- Works for the smallest instance! (Optimality in ~180s)

Initial Solutions

Savings Algorithm

Merge favorable customer pairs in order of how much they "save", relative to a naive initial solution.

Feasible solution not always found, performs well empirically.

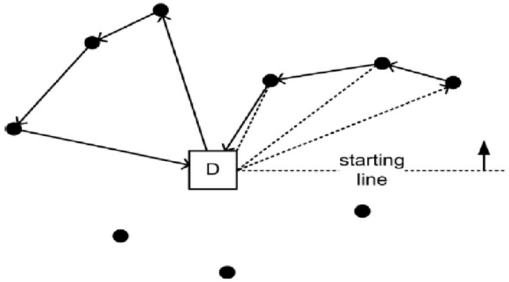


Sweep Algorithm

Create routes in sweeping sequence until trucks are full.

Try numerous starting locations.

Feasible solution not always found, performs poorly empirically.



CP Solver

Let CP solver allocate customers to routes s.t. capacity is satisfied.

Solution always found (if problem is feasible).

Forbid prior solutions.

Local Search Approach

- Local Search
  - Neighborhood of moves: all feasible relocations and swaps.
  - Run until we’re out of time (full 300s search).

- Basic Metaheuristics:
  - Best Improvement, Randomized Iterated Improvement, Simulated Annealing.

Disturbed Best Improvement (best!)

- Use best improvement (hill-climbing) until at a local minimum.
- When we reach a local minimum, jump out using n random moves.
- If we keep returning to past local minima, increase n.
- Initial n depends on number of customers.

Local Exploration Disturbed Best Improvement

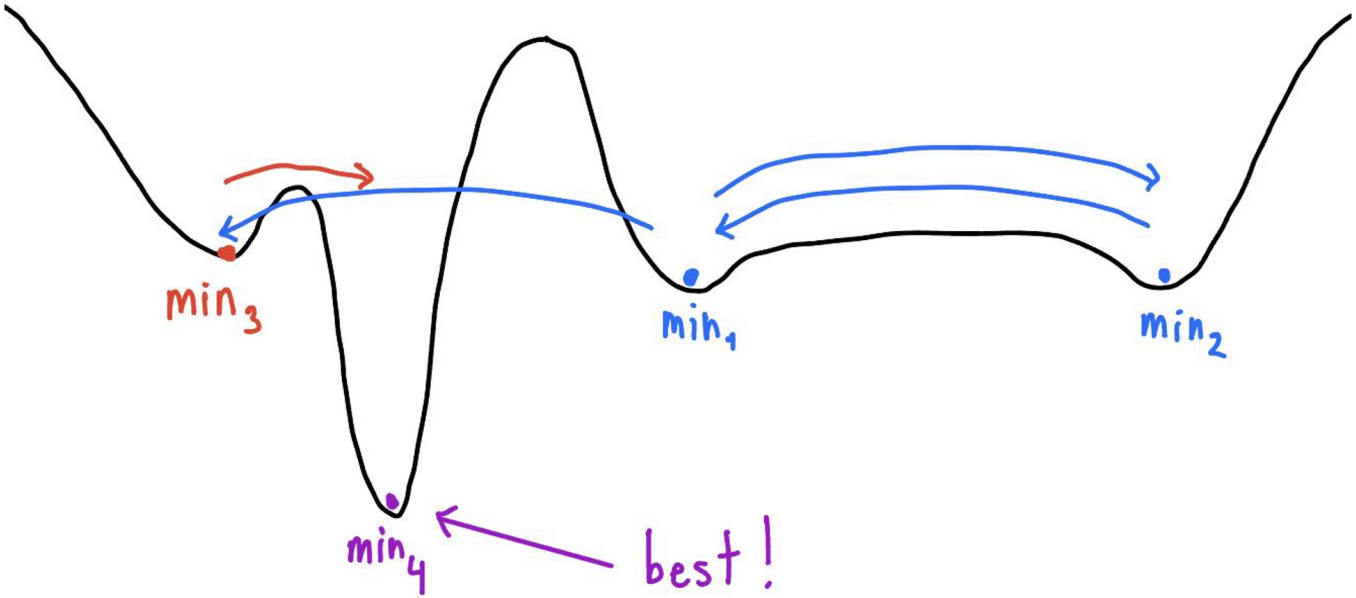
- Setting n depending on the local minimum performed even better!

- Approximation: identify current local minimum by objective value.

- Local exploration parameters:
  - For each local minimum m, explore with parameter n_m.
  - Keep track of what other local minima are reached from m with this n_m.
  - If we keep reaching the same local minima from m, increase n_m.

$n_1 = n_2 = 64$

$n_3 = 32$



Results

- 1 optimal answer (16_5_1.vrp)
- 13/15 Top performing incomplete solutions (top 5, small % off best)

Not great at:

121_7_1.vrp

200_16_2.vrp

(angel_devil unbeatable 👀)

Issues

Cycling in simulated annealing:

- Simulated annealing struggled to find better solutions → Tabu search would have helped.

Getting stuck at local minima:

- Only considered feasible region.
- Only considered (2-opt) neighborhoods.

→ motivation to try disturbance!

Thank you!

Bibliography

Dynamic IP Complete Model inspiration: https://reader.elsevier.com/reader/sd/pii/S0305054898000471?token=07E8C6519635DC9BCC0C0BD55C4DD3A1BAF91821EA2F97F9269378DE37244752F5DA76921C831682A417FF58BBE7D9C6&originRegion=us-east-1&originCreation=20230504171832

Savings Algorithm Original Paper: https://www.jstor.org/stable/167703?seq=3

Savings Algorithm Diagram: https://link.springer.com/article/10.1007/s10845-022-01917-0/figures/4

Sweep Algorithm diagram: https://www.semanticscholar.org/paper/Linear-Sweep-Algorithm-for-Vehicle-Routing-Problem-Kumar-Jayachitra/b08daf63348da7dfcc5971c208c8307b40aca14f/figure/

Simulated Annealing resource: "*Stochastic Local Search: Foundations & Applications (Hoos & Stützle)*" https://www.sciencedirect.com/book/9781558608726/stochastic-local-search

Disturbance Inspiration: Serdar's Lectures! 😄